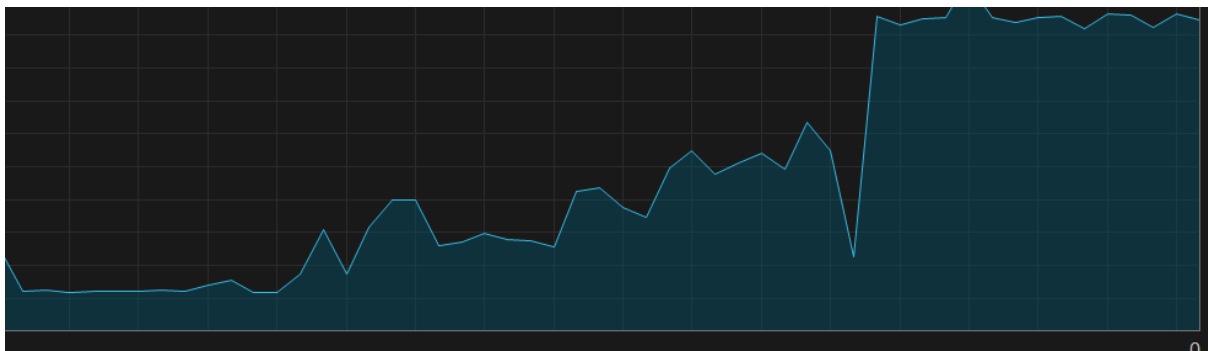


SPRAWOZDANIE 1

Na laboratoriach 1 naszym zadaniem jest mnożenie macierzy o różnych rozmiarach w trybie sekwencyjnym i równoległym. Wykonał kod na tablicach statycznych o wielkości 1024x1024 i uruchomiłem go z liczbą wątków: 1,2,4,8,16,32,64. Po kompilacji i uruchomieniu otrzymałem następujące wyniki:

```
mizula@WIADUKTY:/mnt/c/Users/NeuroSYS$ g++ -std=c++11 -pthread tab1.c
pp
mizula@WIADUKTY:/mnt/c/Users/NeuroSYS$ ./a.out
Liczba watkow:1
23.1176
Liczba watkow:2
11.151
Liczba watkow:4
6.92271
Liczba watkow:8
4.87425
Liczba watkow:16
4.93516
Liczba watkow:32
4.98306
Liczba watkow:64
5.07774
```

Posiadam procesor Intel Core i5-8250U 1.60GHz który ma 4 rdzenie i 8 wątków. Jak widać po rezultatach, 1 wątek czyli wykonanie sekwencyjne uzyskało najgorszy czas. Przy 2 wątkach możemy zobaczyć dwukrotnie lepszy wynik. Analogicznie przy 4 wątkach niemal dwukrotnie lepszy wynik. Przy 8 wątkach wystąpił najlepszy wynik ponieważ wystąpiła pełna wydajność procesora. Każdy rdzeń wykonywał po dwa wątki (łącznie 8). W późniejszych rezultatach widzimy spadek lecz niewielki względem pracy na 8 wątkach. Występuje on ponieważ może procesor działać w pełni lecz zbyt duża ilość wątków powoduje zbyt duży narzut związany z ich przełączaniem oraz pogarsza się użyteczność pamięci cache wątki konkurują, walczą o tę samą przestrzeń podręczną co prowadzi do częstszego jej czyszczenia uwolnienia i ponownego pobrania danych z pamięci Ram.



Powyżej zamieściłem zdjęcie wykorzystania procesora podczas mnożenia macierzy. Jak widać od 8 wątków w górę program wykorzystuje maksymalnie procesor.

Transpozycja a trafienia w cache

Przetestowałem na tablicach statycznych 512x512 dwa warianty jeden ze standardowym mnożeniem macierzy drugi z użyciem transpozycji macierzy B czyli zamiast $c[i][j] += a[i][k] * b[k][j]$ mnożę przez $b_{\text{transponowane}}[j][i]$. Możemy zauważyć, że odczyt poprzez transpozycję jest lepiej wydajny i dąży do poprawy trafień w cache.

Bez transpozycji

```
mizula@WIADUKTY:/mnt/c/Users/NeuroSYS$ ./a.out
Liczba watkow:1
0.81366
Liczba watkow:2
0.402187
Liczba watkow:4
0.300474
Liczba watkow:8
0.205029
Liczba watkow:16
0.200485
Liczba watkow:32
0.199183
Liczba watkow:64
0.197845
```

Z transpozycją

```
mizula@WIADUKTY:/mnt/c/Users/NeuroSYS$ ./a.out
Liczba watkow:1
0.4743
Liczba watkow:2
0.241002
Liczba watkow:4
0.1834
Liczba watkow:8
0.11665
Liczba watkow:16
0.123569
Liczba watkow:32
```

Dynamiczna Tablica a Statyczna

```
int* _t=new int[N*N];
int** T= new int*[N];
for(i=0;i<N;i++){
    T[i]=_t+N*i;
}
```

```
mizula@WIADUKTY:/mnt/c/Users/NeuroSYS$ ./a.out
Liczba watkow:1
25.6257
Liczba watkow:2
12.701
Liczba watkow:4
9.7731
Liczba watkow:8
7.58755
Liczba watkow:16
8.09454
Liczba watkow:32
7.74584
Liczba watkow:64
7.86341
```

Stworzyłem tablice dynamiczne alokując dynamicznie tablice za pomocą tablicy wskaźników tak jak mieliśmy to pokazane na laboratoriach. Po rezultatach widać że na mnożenie macierzy na tablicach dynamicznych jest wolniejsze w niektórych przypadkach nawet do kilku sekund w porównaniu do tablic statycznych.

Reszta Testów

Wszystkie wyniki razem z omówionymi wstawiłem do excela. Najbardziej ciekawym wynikiem według mnie jest przyspieszenie podczas mnożenia dużej tablicy 1024x1024 przy użyciu transpozycji (5 krotnie szybciej). Kod również umieszczam w osobnym pliku.

Typ\Ilość Wątków		2	4	8	16	32	64
Statyczna N=1024	23.1176	11.151	6.92271	4.87425	4.983516	4.98306	5.07774
Statyczna N=512	0.81366	0.402187	0.300474	0.200485	0.205029	0.199183	0.197845
Transpozycja N=1024	3.73789	2.0819	1,20	0.948616	0.941235	1.06537	0.95102
Transpozycja N=512	0.4743	0.241002	0.1834	0.11665	0.1233569	0.133182	0.135888
Dynamiczna N=1024	23.2712	11,74	9,7731	7.58755	8.09454	7.74584	7.86341
Dynamiczna N=512	0.950413	0.470683	0.311239	0.259333	0.231493	0.232127	0.240282
Transpozycja D N=1024	4.36582	2.73574	1.70289	1.10437	1.08499	1.06901	1.07371
Transpozycja D N=512	0.567292	0.293473	0.193681	0.14034	0.1436	0.152438	0.169266